



VERIQTA
SENIOR ENGINEER SERIES

REAL SCENARIOS
REAL THINKING
REAL ENGINEERING
REAL INTERVIEWS

50 DOCKER INTERVIEW SCENARIOS

REAL-WORLD QUESTIONS • PRACTICAL SOLUTIONS • SENIOR-LEVEL THINKING

Build
Debug
Secure
Scale
Deploy



- ✓ CONTAINERS
- ✓ IMAGES
- ✓ NETWORKING
- ✓ STORAGE
- ✓ SECURITY
- ✓ TROUBLESHOOTING
- ✓ PRODUCTION
- ✓ ARCHITECTURE

Think
Like a
Senior Engineer



PRACTICAL
SCENARIOS



REAL-WORLD
PROBLEMS



EXPERT
SOLUTIONS



PRODUCTION
INSIGHTS

FROM INTERVIEW QUESTIONS TO PRODUCTION READINESS

SAME
CONCEPTS
REAL IMPACT

VERIQTA
LEARN | BUILD | GROW

Better
Engineers
Build
Better Systems

CONTAINER STARTS, THEN IMMEDIATELY EXITS

SCENARIOS 1 TO 3

SCENARIO 1

A container starts successfully but exits after two seconds. How do you investigate?

WHAT THIS MAY INDICATE

- Application exits immediately
- Incorrect CMD or ENTRYPOINT
- Missing required environment variables or config
- Process not running in foreground
- PID 1 exits
- Script finishes and container stops

INVESTIGATION STEPS

- 1 Check container status
`docker ps -a`
- 2 View exit code
`docker inspect <container> | grep ExitCode`
- 3 Check logs
`docker logs <container>`
- 4 Inspect CMD and ENTRYPOINT
`docker inspect <container> | grep -A 5 "Config"`
- 5 Run interactively to debug
`docker run -it --entrypoint /bin/sh <image>`

KEY THINGS TO CHECK

- Application command
- Environment variables
- Configuration files
- Entrypoint script logic
- Dependencies
- If the main process stays in the foreground
- Any error message on stderr



SENIOR TIP

A container stays alive only while PID 1 is running. If PID 1 exits, the container stops.

SCENARIO 2

The container exits with code 137. What does that suggest, and what evidence confirms the cause?

WHAT EXIT CODE 137 MEANS

- Exit code 137 = 128 + 9
- Signal 9 (SIGKILL)
- Most common cause: OOMKilled (out of memory)
- Can also be caused by manual kill or system termination

INVESTIGATION STEPS

- 1 Confirm exit code
`docker inspect <container> | grep ExitCode`
- 2 Check if it was OOMKilled
`docker inspect <container> | grep OOMKilled`
- 3 Check container resource usage
`docker stats <container>`
- 4 Review kernel logs on host
`dmesg | grep -i kill`
- 5 Check memory limits
`docker inspect <container> | grep -A 5 "HostConfig"`

EVIDENCE OF OOM

- OOMKilled = true in inspect
- Kernel logs show "Out of memory"
- Memory usage reaches the limit
- Application memory leak
- Container killed by the kernel



SENIOR INSIGHT

Even if the host has free memory, a container can be OOMKilled when it hits its own memory limit.

SCENARIO 3

The application crashes, but docker logs shows almost nothing. What do you inspect next?

POSSIBLE REASONS

- Application crashes before logging
- Logs written to a file instead of stdout/stderr
- Crash happens in native code (no log output)
- Logs buffered and not flushed
- Entrypoint script exits silently
- Permission or path issue for logs

NEXT STEPS

- 1 Inspect container details
`docker inspect <container>`
- 2 Check exit code and error info
`docker inspect <container> | grep -E "ExitCode|Error|OOMKilled"`
- 3 Run container interactively
`docker run -it --entrypoint /bin/sh <image>`
- 4 Check application log files inside container
`docker run -it <image> ls -l /var/log`
- 5 Check system events
`docker events --since 1h`

OTHER THINGS TO CHECK

- Application log file location
- Permissions
- Environment variables
- Crash dumps or core files
- Healthcheck failures
- Any error output on stderr
- Run with higher log level if possible



PRODUCTION NOTE

Do not rely only on docker logs. Use docker inspect, check exit codes, events, and application log files.

CPU AND MEMORY PRODUCTION INCIDENTS**SCENARIOS 4 TO 6**Focus: cgroups • memory limits • CPU quotas • throttling • pressure
docker stats • host vs container resources**SCENARIO 4****One container suddenly consumes 100% CPU.
How do you isolate the cause?****POSSIBLE CAUSES**

- Infinite loop in application code
- High traffic or unexpected load
- CPU-intensive process or thread
- Inefficient algorithm
- Missing resource limits
- Malicious activity (crypto mining)

INVESTIGATION STEPS

- 1 Identify the container
`docker ps --format "table {{.ID}}\t{{.Names}}\t{{.CPUPerc}}"`
- 2 Inspect running processes
`docker exec -it <container> top`
`docker exec -it <container> ps aux --sort=-%cpu`
- 3 Check container resource usage
`docker stats <container>`
- 4 Inspect container configuration
`docker inspect <container> | grep -A 10 "Cpu"`
- 5 Check application logs and recent changes
- 6 If needed, profile the process inside the container

KEY THINGS TO CHECK

- Which process uses the CPU
- Recent deployments or changes
- Traffic patterns
- Resource limits (if any)
- Application logs
- Possible DDoS or abusive traffic

**SENIOR INSIGHT**

High CPU in a container can affect co-located containers on the same host if limits are not set. Always set CPU limits in production.

SCENARIO 5**A container repeatedly gets OOMKilled despite the host having free memory. Why?****COMMON REASONS**

- Container has a low memory limit
- Memory leak in the application
- High memory usage spikes
- Page cache not counted as free
- Multiple containers competing for memory
- Kernel memory or tmpfs usage
- Misinterpreting host free memory

INVESTIGATION STEPS

- 1 Confirm OOMKilled
`docker inspect <container> | grep -A 5 "OOMKilled"`
- 2 Check container memory limits
`docker inspect <container> | grep -A 5 "Memory"`
- 3 Monitor memory usage over time
`docker stats <container>`
- 4 Check application memory usage inside
`docker exec -it <container> free -h`
- 5 Look at kernel messages on the host
`dmesg | grep -i oom`
- 6 Analyze application for memory leaks
`docker exec -it <container> pmap 1 | tail`

KEY CONCEPTS

- Each container has its own cgroup memory limit
- Host free memory does not mean the container can use it
- When limit is exceeded, kernel kills the process (OOMKilled)
- Exit code is usually 137 (128 + 9)

**SENIOR INSIGHT**

In containers, you must monitor the working set, not just total usage. Set realistic memory requests and limits based on production behavior.

SCENARIO 6**Application latency increases whenever CPU usage approaches the container limit. Explain what may be happening.****LIKELY CAUSES**

- CPU throttling due to cgroup quotas
- Not enough CPU allocated
- High context switching
- Competing processes in the same container
- Garbage collection or CPU spikes
- Synchronous operations blocked by CPU starvation

EXPLANATION

- Docker uses CPU quotas (CFS) to limit CPU usage.
- When the container reaches its CPU limit, the kernel throttles the container (stops scheduling its processes) for the remainder of the period.
- This causes increased request latency and timeouts even though the application is still running.
- You will see high throttling metrics in cgroup stats.

CHECK FOR THROTTLING

```
docker stats <container>
docker exec -it <container> cat /sys/fs/cgroup/cpu/cpu.stat
docker inspect <container> | grep -A 5 "CpuQuota"
```

KEY METRICS TO MONITOR

- CPU usage vs CPU limit
- throttled_time
- nr_throttled
- request latency (p95, p99)
- Application queue length
- Error rate and timeouts

**SENIOR INSIGHT**

CPU throttling can look like a network or application issue. Always check cgroup throttling metrics before debugging the application.

CONTAINER NETWORKING FAILURES

Focus: bridge networks • DNS • namespaces • port publishing • bind addresses • network inspection

SCENARIO 7

Two containers on the same host cannot communicate.

COMMON CAUSES

- Containers are on different networks
- Using default bridge network with isolation
- Firewall rules or iptables blocking traffic
- Application listening only on localhost
- Wrong port or service not running
- Custom network driver or policy
- Network namespaces isolation

TROUBLESHOOTING STEPS

- 1 Check networks and container IPs
`docker network ls`
`docker inspect <container> | grep "IPAddress"`
- 2 Verify both containers are on the same network
`docker inspect <container> | grep "Networks"`
- 3 Test connectivity from one container to another
`docker exec -it <container1> ping <container2-ip>`
`docker exec -it <container1> nc -zv <container2-ip> <port>`
- 4 Check if the application is listening on all interfaces
`docker exec -it <container> ss -tlnp`
- 5 Review firewall rules (if any)
`sudo iptables -L -n | grep <port>`

KEY TAKEAWAYS

- Containers must be on the same Docker network to communicate by default.
- Each container has its own network namespace.
- Use a user-defined bridge network for better DNS and communication.
- Verify the application is listening on 0.0.0.0, not just 127.0.0.1.



SENIOR INSIGHT

Use user-defined bridge networks instead of the default bridge network. They provide automatic DNS resolution and better control over network isolation.

SCENARIO 8

A container can reach another container by IP but not by name.

POSSIBLE REASONS

- Not using a user-defined network
- DNS not working or misconfigured
- Using the default bridge network (with no automatic DNS)
- Incorrect container name or alias
- Custom /etc/hosts overriding DNS
- Network driver does not support DNS

INVESTIGATION STEPS

- 1 Check which network the containers are using
`docker inspect <container> | grep "Networks"`
- 2 Test DNS resolution inside the container
`docker exec -it <container> nslookup <container-name>`
`docker exec -it <container> ping <container-name>`
- 3 Inspect Docker's embedded DNS
`cat /etc/resolv.conf`
- 4 Check if the container name is correct
`docker ps --format "table {{.Names}}"`
- 5 Try using a user-defined network
`docker network create app-net`
`docker run --name app1 --network app-net ...`

KEY POINTS

- Automatic DNS resolution works only on user-defined networks.
- Container names act as DNS records within the same network.
- The default bridge network does not provide name resolution.
- You can also define network aliases using `--network-alias`.



SENIOR INSIGHT

In production, always use user-defined networks and meaningful container names or aliases. This simplifies service discovery and debugging.

SCENARIO 9

The application works through localhost inside the container but cannot be reached externally.

COMMON CAUSES

- Application is bound to 127.0.0.1 inside the container
- Port is not published to the host
- Binding to the wrong interface
- Firewall or security group blocking
- Using the wrong host IP or port
- Container is on an internal network only (no external access)

INVESTIGATION STEPS

- 1 Check how the application is listening
`docker exec -it <container> ss -tlnp`
- 2 Verify port publishing
`docker ps --format "table {{.Names}}\t{{.Ports}}"`
- 3 Test from the host
`curl http://localhost:<port>`
`curl http://<host-ip>:<port>`
- 4 Check application bind address (should be 0.0.0.0)
`docker exec -it <container> grep -R "bind" /app/config`
- 5 Inspect container network settings
`docker inspect <container> | grep "Ports"`

KEY TAKEAWAYS

- Use 0.0.0.0 to listen on all interfaces inside the container.
- Publish ports with `-p` or in docker compose.
- Check host firewall rules.
- Ensure you are using the correct host IP and port.
- Remember: localhost inside the container is not the host.



SENIOR INSIGHT

Many "not reachable" issues are caused by the application binding to localhost (127.0.0.1). Always bind to 0.0.0.0 for services that must be accessible externally.

EXTERNAL CONNECTIVITY AND DNS

Focus: Docker DNS • NAT • iptables/nftables • conntrack • ephemeral ports • connection exhaustion • network evidence

SCENARIO 10

Containers suddenly lose internet connectivity while the host remains healthy.

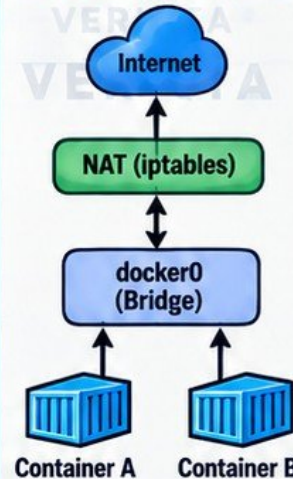
POSSIBLE CAUSES

- NAT or iptables/nftables misconfiguration
- Docker bridge network issue
- DNS issue (upstream or blocked)
- Routing or firewall change
- Exhausted conntrack table
- Network interface or MTU problem
- Security tool blocking container traffic

INVESTIGATION STEPS

- 1 Verify host connectivity
`curl https://example.com`
- 2 Test from container
`docker exec -it <container> curl https://example.com`
- 3 Check Docker networking
`docker network inspect bridge`
- 4 Check NAT and rules
`sudo iptables -t nat -L -n`
or `sudo nft list ruleset`
- 5 Check conntrack
`sudo conntrack -S | head`

NETWORK FLOW (NAT)



KEY POINTS

- Containers use host network for internet access (via NAT).
- Check iptables/nftables MASQUERADE rules.
- Verify DNS and routing.
- Conntrack exhaustion can break new connections.
- Host being healthy does not guarantee container connectivity.

SENIOR INSIGHT

Always compare host network namespace vs container namespace to find where the failure starts.

SCENARIO 11

DNS resolution intermittently fails inside containers.

POSSIBLE CAUSES

- Docker DNS (127.0.0.11) issues
- Upstream DNS server problems
- Network congestion or packet loss
- Misconfigured /etc/resolv.conf
- DNS timeouts or low retries
- Large number of concurrent queries
- Firewall or security tool blocking UDP/TCP 53

INVESTIGATION STEPS

- 1 Check DNS config inside container
`cat /etc/resolv.conf`
- 2 Test DNS manually
`docker exec -it <container> nslookup example.com`
- 3 Try alternatives
`docker exec -it <container> dig example.com`
- 4 Check Docker DNS logs
`sudo journalctl -u docker | grep -i dns`
- 5 Test upstream DNS from host
`dig example.com @8.8.8.8`

KEY POINTS

- Docker uses 127.0.0.11 as an embedded DNS server.
- Intermittent failures can be caused by upstream DNS or network issues.
- Check both UDP and TCP 53.
- Verify DNS search domains and ndots settings.

SENIOR INSIGHT

Intermittent DNS issues often indicate network instability or high query volume. Check DNS metrics and packet loss, not just configuration.

SCENARIO 12

Connections work initially but fail under high traffic.

POSSIBLE CAUSES

- Ephemeral port exhaustion
- Conntrack table full
- NAT gateway port limits
- File descriptor limits
- Upstream service rate limiting
- Network load balancer or firewall limits
- Application not reusing connections (high TCP churn)

INVESTIGATION STEPS

- 1 Check open connections
`ss -s`
- 2 Check conntrack usage
`sudo conntrack -S`
- 3 Check ephemeral port range
`cat /proc/sys/net/ipv4/ip_local_port_range`
- 4 Check for TIME_WAIT buildup
`ss -ant | grep TIME-WAIT | wc -l`
- 5 Monitor during load test
`watch -n 1 "ss -s; conntrack -S"`

KEY POINTS

- Each outbound connection uses an ephemeral port.
- High traffic can exhaust ports or fill the conntrack table.
- TIME_WAIT connections reduce available ports.
- Connection pooling helps reduce churn.

SENIOR INSIGHT

Many "works at low traffic, fails at high traffic" issues are port or conntrack exhaustion. Monitor these metrics in production.

DOCKER STORAGE FAILURES

Focus: writable layers • volumes • bind mounts • UID/GID mapping • overlay storage • disk investigation

SCENARIO 13

Application data disappears after the container is recreated.

COMMON CAUSES

- Data stored in the container's writable layer (not a volume)
- Container was recreated or removed
- Using ephemeral storage
- Incorrect volume mount path
- Named volume not actually mounted
- Using a new container without the existing volume

INVESTIGATION STEPS

- 1 Check how data is stored
`docker inspect <container> | grep Mounts -A 20`
- 2 Inspect volumes
`docker volume ls`
`docker volume inspect <volume>`
- 3 Check container filesystem
`docker exec -it <container> ls -l <path>`
- 4 Verify if a volume was mounted
`docker inspect <container> | grep -A 10 Mounts`
- 5 Recover data if still available
`docker cp <old-container>:/path ./backup`

KEY TAKEAWAYS

- Container writable layer is ephemeral
- Use named volumes for persistent data
- Always verify mounts in production
- Recreating a container without a volume will lose data
- Consider backup strategy for critical data



SENIOR INSIGHT

Containers are disposable. Data must live outside the container lifecycle.

SCENARIO 14

A container cannot write to a mounted directory despite apparently correct permissions.

COMMON CAUSES

- UID/GID mismatch between host and container
- SELinux or AppArmor restrictions
- Read-only mount (ro)
- Filesystem ownership not matching container user
- Running as non-root user
- NFS or network filesystem issues

INVESTIGATION STEPS

- 1 Check current user inside container
`docker exec -it <container> id`
- 2 Check mount details
`docker inspect <container> | grep -A 10 Mounts`
- 3 Verify directory permissions
`docker exec -it <container> ls -ld <path>`
- 4 Check for read-only mount
`docker inspect <container> | grep "RW"`
- 5 Test writing as root (for diagnosis)
`docker exec -it <container> bash`
`touch /path/testfile`

KEY POINTS

- Host permissions and container UID/GID must align
- Volume can be mounted read-only
- SELinux/AppArmor can block writes
- Check filesystem type (e.g. NFS)
- Verify mount options



SENIOR INSIGHT

In productions, avoid running containers as root. Align UID/GID or use proper volume permissions.

SCENARIO 15

Docker consumes almost all available host disk space.

COMMON CAUSES

- Accumulated unused images
- Stopped containers with large writable layers
- Unreferenced volumes
- Build cache and temporary layers
- Logs growing inside containers
- Overlay2 storage consuming space
- No disk cleanup strategy

INVESTIGATION STEPS

- 1 Check disk usage by Docker objects
`docker system df`
- 2 Identify large directories
`sudo du -sh /var/lib/docker/*`
- 3 Check running and stopped containers
`docker ps -a --size`
- 4 Check unused images, containers, volumes
`docker image ls -a`
`docker container ls -a`
`docker volume ls`
- 5 Clean up safely (after verification)
`docker system prune -a`
`docker volume prune`

KEY POINTS

- Monitor disk usage regularly
- Use log rotation for container logs
- Remove unused images and containers
- Be careful with docker system prune
- Consider setting storage quotas
- Use external logging for application logs



SENIOR INSIGHT

Disk exhaustion can bring down the entire Docker host. Implement monitoring and automated cleanup policies before it becomes an incident.

PERSISTENT DATA AND STATEFUL CONTAINERS

Focus: lifecycle separation • volume inspection • backup • recovery • state ownership

SCENARIO 16

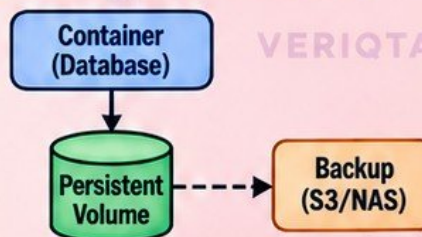
A database container must survive replacement without losing data.
How would you design it?

KEY CONSIDERATIONS

- Use persistent volumes (not container writable layer)
- Separate data from container lifecycle
- Choose the right storage driver (local, NFS, cloud block storage)
- Implement regular backups
- Use reliable restart and recovery procedures
- Consider high availability and replication
- Define clear ownership of the data

POSSIBLE DESIGN

- 1 Use a Docker volume or external storage (EBS/NFS) for database data
- 2 Configure the container to use the volume (e.g. `-v db-data:/var/lib/mysql`)
- 3 Store configuration separately if needed
- 4 Implement automated backups (docker exec or external tools)
- 5 Use health checks and restart policies
- 6 For high availability, use replication or a managed database service



USEFUL COMMANDS

- `docker volume create db-data`
- `docker run -d --name db \`
`-v db-data:/var/lib/mysql ...`
- `docker inspect db-data`
- `docker exec -it db mysqldump ...`
- `docker logs db`
- `docker ps -a`
- `docker volume ls`

KEY TAKEAWAYS

- Containers are replaceable, data is not.
- Always use persistent storage for databases.
- Automate backups and test restores regularly.
- Design for failure and plan recovery.
- Clearly define who owns the data.

SCENARIO 17

A volume contains critical production data but its container is corrupted.
How do you recover safely?

INITIAL STEPS

- Do not delete the volume.
- Stop and preserve the corrupted container for investigation.
- Identify the volume name and mount point.
- Create a new clean container and mount the existing volume.
- Verify the data integrity before restarting the application.
- Take a backup before making any changes.

RECOVERY STEPS

- 1 List existing volumes
`docker volume ls`
- 2 Inspect the volume
`docker volume inspect <volume>`
- 3 Start a new container and mount the volume
`docker run -it --rm -v <volume>:/data alpine`
- 4 Verify the data (check files, permissions, integrity)
- 5 If needed, copy the data to a new volume
`docker run -it --rm -v <volume>:/data \`
`-v new-volume:/backup alpine \`
`cp -r /data/* /backup/`
- 6 Once verified, deploy a new container using the existing (or new) volume

THINGS TO CHECK

- Volume contains expected data
- File permissions and ownership (UID/GID)
- Application data is not corrupted
- Mount options (read-only/read-write)
- Filesystem integrity (fsck if needed)
- Application logs for consistency
- Take a backup before making changes

KEY TAKEAWAYS

- Volumes can be reused independently of containers.
- Always verify data before restarting in production.
- Keep backups and have a recovery plan.
- Container failure should not mean data loss.

IMAGE SIZE AND LAYER PROBLEMS

VERIQTA Focus: layers • build context • caching • multi-stage builds • .dockerignore • immutable image history

SCENARIO 18

A production image is 2 GB. How do you determine why?

POSSIBLE CAUSES

- Large base image
- Unnecessary packages and tools
- Source code and build artifacts included in the image
- Missing .dockerignore
- Multiple large layers
- Package manager caches not cleaned
- Logs and temporary files
- Not using multi-stage builds
- Copying the entire build context

INVESTIGATION STEPS

- 1 Check overall image size
`docker images`
- 2 Inspect layer details
`docker history <image>`
- 3 Inspect image contents
`docker run --rm -it <image> sh`
- 4 Look for large directories
`du -sh /*`
- 5 Review Dockerfile and .dockerignore

USEFUL COMMANDS

```
docker images
docker history <image>
docker run --rm -it <image> sh
docker build --no-cache .
```

KEY TAKEAWAYS

- Use minimal base images (e.g. alpine, distroless)
- Remove build tools and caches
- Use .dockerignore to exclude unnecessary files
- Use multi-stage builds
- Keep only what is needed at runtime
- Regularly review image layers



SENIOR INSIGHT

Image size impacts security, scan time, deployment speed and cost. A smaller image is easier to maintain and less likely to contain vulnerabilities.

SCENARIO 19

A secret was deleted in a later Dockerfile instruction. Is it actually gone?

WHY IT HAPPENS

- Docker images are immutable
- Each instruction creates a new layer
- Deleting a file in a later layer does not remove it from previous layers
- Secrets remain in image history
- Anyone with access can extract it
- Common with API keys, credentials and tokens

INVESTIGATION STEPS

- 1 Check image history
`docker history <image>`
- 2 Inspect layers for the secret
`docker run --rm <image> sh`
`grep -R "secret" /`
- 3 Use a tool to analyze layers
`docker save <image> | tar -tv`
- 4 Check the Dockerfile for how the secret was added
- 5 Verify if the secret is still accessible

BEST PRACTICES

- Never add secrets using COPY or ADD in a Dockerfile
- Use BuildKit secrets instead
- Use environment variables or external secret managers
- Use multi-stage builds
- Scan images for secrets



SENIOR INSIGHT

Once a secret is in a layer, it is part of the image forever, even if you delete it later. Treat images as public and never bake secrets into them.

SCENARIO 20

A tiny source-code change causes an expensive dependency layer to rebuild.

COMMON CAUSES

- COPY . . placed before dependency installation
- Build context includes unnecessary files
- No .dockerignore (e.g. node_modules)
- Layer cache invalidated by file changes
- Changing file metadata (timestamps)
- Using a single-stage build
- Non-deterministic builds

TROUBLESHOOTING STEPS

- 1 Review the Dockerfile layer order
- 2 Check what files are in the build context
`docker build . --no-cache (check output)`
- 3 Verify .dockerignore is configured
- 4 Reorder COPY and RUN instructions
- 5 Use multi-stage builds and cache mounts
- 6 Confirm the cache is working
`docker build --progress=plain .`

KEY TAKEAWAYS

- Copy dependency files first (e.g. package.json, requirements.txt)
- Install dependencies before copying application code
- Use .dockerignore to reduce context size
- Use multi-stage builds
- Take advantage of build cache



SENIOR INSIGHT

Docker layer caching is powerful but sensitive to file changes. Structure your Dockerfile to maximize cache reuse and keep builds fast and reliable.

DOCKERFILE PRODUCTION REVIEW

Focus: reproducibility • least privilege • build hygiene • attack surface • dependency control

SCENARIO 21

Review a Dockerfile that runs everything as root.

RISKS OF RUNNING AS ROOT

- Larger blast radius if container is compromised
- Attacker gets full control inside the container
- Can modify system files and installed packages
- Violates least privilege principle
- Increases lateral movement risk in case of container escape

WHAT TO REVIEW

- 1 Check for USER instruction (dockerfile)
- 2 Identify commands that require root (e.g. package installation)
- 3 Create a non-root user and group
- 4 Set correct ownership for application files
- 5 Use least privilege (only elevate when necessary)

BETTER EXAMPLE

```
FROM python:3.11-slim
RUN useradd -m appuser
WORKDIR /app
COPY . .
RUN chown -R appuser:appuser /app
USER appuser
```

KEY TAKEAWAYS

- Do not run containers as root.
- Use a dedicated non-root user.
- Only use root for specific steps (e.g. installing packages).
- Set proper file ownership.
- Follow the principle of least privilege.
- Improves security and reduces impact of compromise.



SENIOR INSIGHT

Many real-world container breaches start with a process running as root. Always assume the container can be compromised and minimize the damage.

SCENARIO 22

COPY . . causes security and build-efficiency problems. What would you change?

PROBLEMS WITH COPY . .

- Copies unnecessary files (e.g. .git, tests, local configs)
- Increases image size
- Slows down builds
- May include sensitive files (e.g. .env, SSH keys, credentials)
- Invalidates cache frequently when any file changes
- Exposes source code that should not be in the image

HOW TO IMPROVE

- 1 Use a .dockerignore file
- 2 Copy only what is needed
- 3 Order COPY commands to maximize cache reuse
- 4 Separate dependency files from source code
- 5 Keep sensitive files out of the image

EXAMPLE .dockerignore

```
.git
.env
node_modules
*.log
tests/
```

BETTER COPY PATTERN

```
COPY package.json ./
RUN npm install
COPY src/ ./src/
```

KEY TAKEAWAYS

- Use .dockerignore to exclude unnecessary files.
- Copy only what you need.
- Structure COPY for better cache usage.
- Keep sensitive files out of the build context.
- Smaller, faster, and more secure images.



SENIOR INSIGHT

A large build context can leak secrets and significantly slow down CI/CD. Always review what you are copying into the image.

SCENARIO 23

A team installs packages without pinning versions. What production risk does this introduce?

RISKS OF UNPINNED VERSIONS

- Non-reproducible builds
- Unexpected breaking changes
- Security vulnerabilities in newer versions
- Application behaves differently over time
- Harder to debug and roll back
- Can break builds when upstream packages are updated

HOW TO FIX

- 1 Pin exact package versions
- 2 Use version ranges only when appropriate
- 3 Use lock files (e.g. requirements.txt, package-lock.json, go.mod)
- 4 Regularly update and test dependencies
- 5 Consider using a private package registry for stability

EXAMPLE (Python)

```
RUN pip install flask==2.3.3
# instead of: pip install flask
```

KEY TAKEAWAYS

- Always pin package versions.
- Use lock files for dependency control.
- Regularly update and test for vulnerabilities.
- Be intentional about version changes.
- Ensures reproducible, secure, and stable builds.



SENIOR INSIGHT

Unpinned dependencies can break your application without any code change. Reproducible builds are essential for reliable production systems.

ENTRYPOINT, CMD AND PID 1

Focus: PID 1 semantics • signal propagation • shell versus exec form • graceful termination

SCENARIO 24

A container ignores docker stop and eventually gets killed.

POSSIBLE CAUSES

- PID 1 is not handling signals
- Using shell form (e.g. /bin/sh -c)
- Signal not propagated to child processes
- Application does not trap SIGTERM
- Long shutdown process exceeding stop timeout
- Zombie processes not reaped
- Using a wrapper script without exec

INVESTIGATION STEPS

- 1 Check the container's PID 1
`docker top <container>`
- 2 Inspect the entrypoint and cmd
`docker inspect <container> | grep -i -A5 "Entrypoint"`
- 3 Check process tree
`docker exec <container> ps -ef`
- 4 Test signal handling manually
`docker exec <container> kill -TERM 1`
- 5 Review application logs for shutdown behavior

KEY TAKEAWAYS

- PID 1 must handle SIGTERM.
- Use exec form or a proper init process (tini, dumb-init).
- Ensure the application traps and handles signals.
- Set an appropriate stop timeout if needed.
- Verify graceful shutdown in production-like testing.



SENIOR INSIGHT

PID 1 behaves differently. If it does not handle signals, Docker will eventually send SIGKILL, which can cause data loss and corruption.

SCENARIO 25

An application does not receive SIGTERM during deployment.

POSSIBLE CAUSES

- Using shell form ENTRYPOINT/CMD
- Signal not forwarded to the actual application
- Wrapper script missing exec
- Multiple processes with PID 1 not forwarding signals
- Orchestrator sending SIGTERM but application not listening
- Language runtime not handling signals by default

INVESTIGATION STEPS

- 1 Inspect Dockerfile for shell form (e.g. ENTRYPOINT /bin/sh -c "...")
- 2 Check PID 1 and process tree
`docker exec <container> ps -ef`
- 3 Send SIGTERM manually
`docker exec <container> kill -TERM 1`
- 4 Check application logs for signal handling
- 5 Verify orchestrator configuration (e.g. Kubernetes terminationGracePeriodSeconds)

COMMON FIXES

- Use exec form: ENTRYPOINT ["app"]
- Add exec in wrapper scripts:
`exec "$@"`
- Use a minimal init system (tini)
- Ensure the application handles SIGTERM
- Test graceful shutdown locally
- Set proper termination grace period in orchestrators



SENIOR INSIGHT

Always test signal handling in a real container. Many applications work fine locally but ignore SIGTERM in containers due to PID 1 and shell wrappers.

SCENARIO 26

Explain a production failure caused by incorrect ENTRYPOINT and CMD interaction.

EXAMPLE SCENARIO

A container was built with:

```
ENTRYPOINT ["/docker-entrypoint.sh"]
CMD ["app"]
```

During deployment, the team overrode the command:

```
docker run image --config
```

The script did not use `exec "$@"`, so the CMD arguments were ignored. The application started with default configuration, causing a misconfigured service in production.

WHAT WENT WRONG

- 1 ENTRYPOINT script did not pass CMD arguments correctly.
- 2 Shell form or missing exec caused arguments to be ignored.
- 3 The application started without expected configuration.
- 4 Service ran with defaults, leading to failures (e.g. wrong database, no TLS).

CORRECT APPROACH

```
# entrypoint.sh
#!/bin/sh
exec "$@"

Or use exec form directly:
ENTRYPOINT ["app"]
CMD ["--config"]
```

KEY TAKEAWAYS

- Understand how ENTRYPOINT and CMD combine.
- Use exec to pass arguments.
- Prefer exec form over shell form.
- Test different run command overrides.
- Verify final command in a running container: `docker inspect`.
- Misconfiguration can cause real production outages.



SENIOR INSIGHT

ENTRYPOINT and CMD flexibility is powerful but dangerous. Always document the intended usage and test command overrides to avoid surprises in production.

CONTAINER HEALTH AND RESTART LOOPS

Focus: health checks • restart policies • dependency failures • readiness versus process existence • investigation order

SCENARIO 27

A container is running but the application is unusable.

POSSIBLE CAUSES

- Application process is running but not healthy
- Dependency failure (database, cache, external API)
- Application misconfiguration
- Port is open but service not ready
- Health check not configured
- High resource usage (CPU/memory)
- Application stuck in a bad state
- Network or DNS issues

INVESTIGATION STEPS

- 1 Check container status
`docker ps`
- 2 View logs for application errors
`docker logs <container>`
- 3 Check application health endpoint
(e.g. `curl http://localhost:8080/health`)
- 4 Verify dependencies (DB, cache, APIs)
- 5 Inspect resource usage
`docker stats <container>`
- 6 Check configuration and environment variables

KEY TAKEAWAYS

- Running does not mean healthy.
- Check logs and dependencies first.
- Implement proper health checks.
- Monitor resources and external services.
- Use readiness checks for real application health.



SENIOR INSIGHT

A container can be "up" but still failing user requests. Always verify the full dependency chain, not just the container process.

SCENARIO 28

A container continuously restarts every few seconds.

POSSIBLE CAUSES

- Application exits with an error
- Misconfiguration or missing files
- Dependency not available (e.g. database)
- Crash due to memory or CPU limits
- Incorrect CMD or ENTRYPOINT
- Health check failure with restart policy
- Application bug or unhandled exception

INVESTIGATION STEPS

- 1 Check restart count
`docker ps -a`
- 2 Inspect container logs
`docker logs <container>`
- 3 Check exit code
`docker inspect <container> | grep "ExitCode"`
- 4 Review application configuration
- 5 Verify dependencies and environment variables
- 6 Run container interactively to debug
`docker run -it --entrypoint sh <image>`

KEY TAKEAWAYS

- Check logs and exit code first.
- Identify the root cause, not just increase restart limits.
- Fix the underlying issue instead of masking with restart policies.
- Use restart policies appropriately (e.g. on-failure, unless-stopped).
- Test locally before deploying changes.



SENIOR INSIGHT

A restart loop is a symptom, not the real problem. Find and fix the cause instead of just setting "restart: always."

SCENARIO 29

A health check reports failure even though the application responds manually.

POSSIBLE CAUSES

- Health check command is incorrect
- Health check is checking the wrong endpoint or port
- Timing issues (service not ready yet)
- Different environment inside the container (e.g. localhost vs 127.0.0.1)
- Health check has too short timeout or interval
- Application requires authentication
- Health check depends on external service that is slow or unavailable

INVESTIGATION STEPS

- 1 Inspect health check configuration
`docker inspect <container> | grep -A 10 "Health"`
- 2 Manually run the health check command inside the container
- 3 Compare environment (user, PATH, network)
- 4 Check logs for health check failures
`docker logs <container>`
- 5 Adjust timeout, interval, or start period if needed
- 6 Verify if the endpoint requires authentication or specific headers

KEY TAKEAWAYS

- Manual testing and health check may run in different contexts.
- Use the same command as the health check to test.
- Check timing, authentication, and dependencies.
- Tune health check parameters (interval, timeout, retries).
- Ensure health check validates real application readiness.



SENIOR INSIGHT

A health check should test true readiness, not just process existence. Design it to reflect real user traffic and dependency health.

DOCKER SECURITY INCIDENTS

Focus: capabilities • namespaces • privileged containers • Docker daemon access • host compromise paths

SCENARIO 30

An attacker compromises an application running inside a container.
What determines the potential blast radius?

FACTORS THAT DETERMINE THE BLAST RADIUS

- Container privileges (root vs non-root)
- Linux capabilities (e.g. --cap-add)
- Namespace isolation (PID, network, mount, user, etc.)
- Access to sensitive host paths (e.g. /var/run/docker.sock, /etc, /proc)
- Network access (internal systems, metadata services, databases)
- Secrets and credentials inside container
- Lateral movement possibilities (to other containers or the host)
- Security controls (AppArmor, SELinux, seccomp, read-only filesystem)

INVESTIGATION STEPS

- 1 Check container ID and inspect config
`docker inspect <container>`
- 2 View running processes and user
`docker exec <container> ps aux`
- 3 Check capabilities and security profile
`docker inspect <container> | grep -i Cap`
- 4 Check mounted volumes
`docker inspect <container>`
- 5 Review network access
`docker network inspect <network>`
- 6 Look for signs of lateral movement (logs, connections, modified files)

KEY TAKEAWAYS

- Blast radius depends on isolation and privileges.
- Use least privilege (non-root, minimal capabilities).
- Restrict network and filesystem access.
- Protect secrets and credentials.
- Use security profiles (AppArmor/SELinux/seccomp).
- Monitor for unusual behavior.



SENIOR INSIGHT

Containers are not a security boundary if misconfigured. A compromised container can often lead to host compromise when granted excessive privileges or access to sensitive resources.

SCENARIO 31

A container requires --privileged. What questions should you ask before approving it?

QUESTIONS TO ASK

- Why is --privileged required? (use case and justification)
- Can the requirement be met with specific capabilities instead?
- What devices or kernel features are needed?
- Is there a safer alternative (e.g. --cap-add, device mounts)?
- What is the risk to the host and other containers?
- Is this running in a trusted environment?
- How will it be monitored and audited?
- Is there a time limit or review process?

SAFER ALTERNATIVES

- 1 Use specific capabilities
`--cap-add=<capability>`
- 2 Mount only required devices
`--device /dev/specific`
- 3 Use read-only filesystem
`--read-only`
- 4 Use security profiles (AppArmor/SELinux/seccomp)
- 5 Run as non-root user
- 6 Isolate with dedicated host or VM if required

KEY TAKEAWAYS

- --privileged gives almost all host capabilities.
- Only approve if absolutely necessary.
- Prefer minimal capabilities and device mounts.
- Document and review the requirement.
- Apply additional controls and monitoring.



SENIOR INSIGHT

If a container truly needs --privileged, it may be a sign that it is doing something that should run on the host, not inside a container. Challenge the requirement and look for a safer design.

SCENARIO 32

Developers mount /var/run/docker.sock into an application container. Why is this dangerous?

WHY IT IS DANGEROUS

- Gives full control of the Docker daemon
- Allows attackers to create, modify, and delete containers
- Can be used to mount host filesystems
- Can escape the container and gain host access
- Can access sensitive data and secrets from other containers
- Bypasses container isolation completely

REAL ATTACK EXAMPLE

- 1 Attacker gains code execution in container
Uses Docker socket to run a privileged container on the host
`docker run -v /:/host --privileged alpine`
- 2 Chroots into the host filesystem
`docker exec -it <id> chroot /host`
- 3 Accesses host files, credentials, and services
- 4 Achieves full host compromise

KEY TAKEAWAYS

- Never mount /var/run/docker.sock into application containers.
- Use the Docker API with proper authentication and authorization if needed.
- Consider rootless Docker or isolated build systems.
- Restrict access to the Docker daemon.
- Monitor and audit Docker API usage.



SENIOR INSIGHT

Mounting the Docker socket is equivalent to giving root access to the host. It completely defeats container security.

SECRETS AND SUPPLY-CHAIN SECURITY

Focus: BuildKit secrets • scanning • SBOMs • digests • provenance • signing • immutable artifacts

SCENARIO 33

Credentials are discovered inside an image pushed to the registry.

HOW DOES THIS HAPPEN?

- Hardcoded credentials in Dockerfile or source code
- Using ARG or ENV for sensitive data
- Copying .env or config files into the image
- Secrets left in build layers (even if later deleted)
- Using legacy build instead of BuildKit secrets
- Accidentally committing credentials to version control
- Pulling base images that already contain secrets

INVESTIGATION STEPS

- 1 Inspect image history
`docker history <image>`
- 2 Scan image for secrets
`trivy image <image>`
`# or gitleaks / detect-secrets`
- 3 Inspect filesystem contents
`docker run --rm -it <image> sh`
- 4 Check environment variables and config files
- 5 Review Dockerfile and build logs
- 6 Rotate and revoke any exposed credentials immediately

KEY TAKEAWAYS

- Never use ARG or ENV for secrets.
- Use BuildKit secrets (`--secret`) instead.
- Do not copy .env or sensitive files into the image.
- Scan images in CI before pushing.
- Use secret scanning tools.
- Rotate exposed credentials immediately.
- Implement pre-commit hooks (e.g. gitleaks).
- Follow least privilege.



SENIOR INSIGHT

Secrets in image layers can still be recovered even if deleted in later layers. Assume anything in a pushed image is public. Rotate immediately and investigate how it happened.

SCENARIO 34

Your approved base image receives a critical CVE.

IMPACT AND RISKS

- Known vulnerabilities can be exploited in your images.
- Affects all downstream images.
- May violate compliance requirements.
- Increases risk of container escape or lateral movement.
- Can impact availability and trust.
- Requires urgent patching and rebuilds.

RESPONSE STEPS

- 1 Identify affected images
(trivy, gype, docker scout)
- 2 Check CVE details and severity (CVSS score)
- 3 Look for patched base image version
- 4 Rebuild and test your images
- 5 Scan again to confirm the fix
- 6 Redeploy and monitor

KEY TAKEAWAYS

- Regularly scan base images.
- Subscribe to security advisories.
- Use minimal and trusted base images (e.g. distroless, alpine).
- Automate rebuilds when new versions are released.
- Keep a clear inventory of all base images in use.
- Integrate vulnerability scanning in CI/CD.



SENIOR INSIGHT

CVE management is a continuous process. Automate detection and rebuilds. A patched base image is not enough — you must rebuild, scan, and deploy to actually be safe.

SCENARIO 35

How would you establish that the image deployed to production is the artifact that passed CI?

KEY CONCEPTS

- Immutable artifacts
- Image digests (SHA256)
- SBOMs (Software Bill of Materials)
- Provenance (build metadata)
- Image signing (cosign)
- Secure registry and access control
- Traceability from source to production
- Policy enforcement (admission controls)

IMPLEMENTATION STEPS

- 1 Build and push image with immutable tag and record its digest
`docker build -t app:1.0.0 .`
`docker push app@sha256:<digest>`
- 2 Generate and store SBOM
`syft <image> -o spdx-json`
- 3 Sign the image
`cosign sign <image>`
- 4 Store provenance (SLSA, in-toto)
- 5 Verify the same digest is deployed (in Kubernetes, Helm, or your platform)
- 6 Enforce verification with policy (e.g. Kyverno, OPA, admission controller)

KEY TAKEAWAYS

- Use immutable digests, not mutable tags like latest.
- Generate SBOMs for all images.
- Sign images and verify signatures in production.
- Store build provenance (SLSA level).
- Use an artifact registry with access control.
- Implement policy to only allow signed and verified images.
- Maintain full traceability from commit to production.



SENIOR INSIGHT

Tags can be changed, but digests cannot. Use signing and provenance to ensure supply-chain integrity. If you cannot prove where the image came from, assume it is untrusted.

DOCKER BUILD FAILURES

Focus: build context • cache invalidation • architecture differences • dependencies • reproducible builds • BuildKit

SCENARIO 36

docker build works on one engineer's machine but fails in CI.

POSSIBLE CAUSES

- Different Docker versions or BuildKit settings
- Missing files in build context (e.g. .dockerignore)
- Different base image availability or tags
- Architecture mismatch (arm64 vs amd64)
- Private registry or network access issues in CI
- Environment variables not set in CI
- Hidden dependencies (e.g. git, ssh, certs)

INVESTIGATION STEPS

- 1 Verify Docker version and BuildKit settings in both environments (docker version, docker buildx inspect).
- 2 Check build context contents (use docker build --no-cache -f /dev/null).
- 3 Compare .dockerignore files and excluded files.
- 4 Check base image availability and platform (docker pull).
- 5 Verify network access and registry authentication (docker login, ping).
- 6 Reproduce with docker build --progress=plain to see detailed errors.

KEY TAKEAWAYS

- Keep Docker versions consistent across environments.
- Use a proper .dockerignore file.
- Ensure base images are accessible in CI.
- Check for architecture differences.
- Use BuildKit for better logs and reproducibility.
- Make builds reproducible and environment independent.



SENIOR INSIGHT

If it works locally but fails in CI, assume an environment difference until proven otherwise. Reproduce the CI environment locally when possible.

SCENARIO 37

Builds suddenly become dramatically slower despite minimal application changes.

POSSIBLE CAUSES

- Build cache invalidation (e.g. changed files early in Dockerfile)
- Large build context (unnecessary files)
- Dependency updates (e.g. apt, npm, pip)
- Base image pulled again (no cache)
- Multi-architecture builds (e.g. buildx, QEMU)
- BuildKit not using cache effectively
- Network or registry latency
- Changes in build args or environment variables

INVESTIGATION STEPS

- 1 Check what changed in the Dockerfile and build context (git diff, ls -la).
- 2 Verify which layers are being rebuilt (docker build --progress=plain).
- 3 Check cache usage (docker build --no-cache vs normal).
- 4 Review build context size (use docker build --progress=plain).
- 5 Check for dependency updates (apt, npm, pip, yarn).
- 6 Compare build times with and without BuildKit (DOCKER_BUILDKIT=1).

KEY TAKEAWAYS

- Keep the build context small and clean.
- Use proper layer ordering (cache-friendly).
- Pin dependencies for stability and faster builds.
- Leverage BuildKit and remote cache.
- Use multi-stage builds.
- Avoid unnecessary changes in early layers.



SENIOR INSIGHT

Small changes can invalidate large caches. Structure your Dockerfile and build context to maximize cache hits and predictability.

USEFUL COMMANDS & EXAMPLES

Check build context size:

```
docker build --progress=plain .
du -sh .
```

Use .dockerignore:

```
node_modules/
*.log
.git/
```

View cache usage:

```
docker build --no-cache .
docker buildx build --progress=plain .
```

Enable BuildKit:

```
export DOCKER_BUILDKIT=1
docker build .
```



PRO TIPS

- ✓ Pin base images with digests (e.g. ubuntu@sha256:...)
- ✓ Use multi-stage builds to reduce size.
- ✓ Keep dependencies locked (e.g. requirements.txt, package-lock.json).
- ✓ Use a remote cache for CI (e.g. registry cache).
- ✓ Monitor build times and cache hit ratio.

MULTI-ARCHITECTURE CONTAINERS

Focus: ● architecture compatibility ● manifests ● emulation ● Buildx ● multi-platform pipelines

SCENARIO 38

An image built on Apple Silicon fails on an AMD64 production server.

POSSIBLE CAUSES

- Image was built for ARM64 (Apple Silicon) but deployed on AMD64.
- Using `--platform` not specified.
- Base image or dependencies are architecture-specific.
- Compiled binaries for ARM64 (e.g. Go, Rust, C) run on AMD64.
- Using local build cache with Apple Silicon layers.
- CI/CD pipeline not building for the correct architecture.
- Missing multi-arch support in the Dockerfile.

INVESTIGATION STEPS

- 1 Inspect the image architecture
`docker image inspect <image> | grep Architecture`
- 2 Check your local architecture
`uname -m` (arm64 or x86_64)
- 3 Check image manifest
`docker manifest inspect <image>`
- 4 Test running with explicit platform
`docker run --platform linux/amd64 <image>`
- 5 Review Dockerfile for platform-specific instructions or binaries
- 6 Rebuild for the correct platform using Buildx (see below)

KEY TAKEAWAYS

- Apple Silicon (ARM64) and AMD64 are different architectures.
- Use multi-architecture images or build for the target platform.
- Always test images on the same architecture as production.
- Use Buildx for cross-platform builds.
- Avoid architecture-specific binaries unless necessary.



SENIOR INSIGHT

Many official images are multi-arch, but your application and its dependencies may not be. Always verify the architecture and test on a clean environment that matches production.

SCENARIO 39

Your organization must publish the same application for ARM64 and AMD64.

REQUIREMENTS

- Build images for both ARM64 and AMD64.
- Publish a single image reference with a multi-architecture manifest.
- Ensure the correct image is pulled automatically based on the host architecture.
- Integrate into CI/CD pipeline.

USEFUL COMMANDS

```
# Check local architecture
uname -m

# Inspect image architecture
docker image inspect <image> | grep Architecture

# Inspect multi-arch manifest
docker manifest inspect <image>

# Create and use Buildx builder
docker buildx create --use

# Build and push multi-arch image
docker buildx build --platform linux/amd64,linux/arm64 \
-t <registry>/<app>:<tag> --push
```

IMPLEMENTATION STEPS

- 1 Set up Docker Buildx
`docker buildx create --use`
- 2 Build and push multi-arch image
`docker buildx build --platform linux/amd64,linux/arm64 \ -t <registry>/<app>:<tag> --push`
- 3 Verify the manifest
`docker manifest inspect \ <registry>/<app>:<tag>`
- 4 Test on different architectures
`docker run --rm <image> uname -m`
- 5 Automate in CI/CD (GitHub Actions, GitLab CI, etc.)
- 6 Monitor and update base images for both architectures

KEY TAKEAWAYS

- Use Docker Buildx for multi-platform builds.
- A single image tag can contain multiple architectures (manifest list).
- Clients automatically pull the correct image for their architecture.
- Test images on both ARM64 and AMD64.
- Keep builds reproducible and automated.



SENIOR INSIGHT

Multi-architecture support is essential for modern infrastructure (cloud, edge, ARM-based instances). Build once, test both, and push a single multi-arch image to avoid maintenance headaches.

MULTI-ARCHITECTURE FLOW



SUPPORTED PLATFORMS (EXAMPLES)

- linux/amd64 (x86_64)
- linux/arm64 (ARM64)
- linux/arm/v7 (ARMv7)
- linux/arm/v6 (ARMv6)
- linux/386 (32-bit x86)
- linux/s390x (IBM Z)
- linux/ppc64le (PowerPC)

DOCKER HOST FAILURE

Focus: daemon architecture • containerd • runtime relationships • daemon logs • systemd • host investigation • recovery risk

SCENARIO 40

Docker commands hang while running containers continue serving traffic.

WHAT IS HAPPENING?

- The Docker daemon (dockerd) is not responding.
- Running containers continue to serve traffic because they are managed by containerd and runc, not directly by the Docker CLI.
- The CLI cannot communicate with the daemon via the Docker socket (/var/run/docker.sock).
- This is a control-plane issue, not necessarily a container runtime failure.

INVESTIGATION STEPS

- 1 Check if the Docker daemon is running
`systemctl status docker`
- 2 Check Docker daemon logs
`journalctl -u docker -n 100 --no-pager`
- 3 Check if the Docker socket is accessible
`ls -l /var/run/docker.sock`
- 4 Verify containerd status
`systemctl status containerd`
- 5 Check running containers (even if docker CLI hangs, try crictl or containerd tools)
`ctr -n moby containers ls`
- 6 Check system resources (CPU, memory, disk, inode)
`free -h && df -h && top`
- 7 Identify recent system changes (dmesg, syslog, kernel messages)
`dmesg | tail -n 50`

KEY TAKEAWAYS

- Containers can keep running even if the Docker daemon is down.
- Docker CLI communicates with dockerd via a Unix socket.
- containerd and runc continue managing running containers.
- Always monitor daemon health, not just container health.
- Set up alerting for Docker daemon availability.
- Understand the separation between control plane (daemon) and data plane (containers).
- Have a recovery plan for daemon failures.



SENIOR INSIGHT

A healthy production setup monitors the Docker daemon, containerd, and the host system. Containers running does not mean the Docker engine is healthy. You can lose the ability to manage, update, or recover containers if the daemon is down.

SCENARIO 41

The Docker daemon unexpectedly restarts on a production host.

POSSIBLE CAUSES

- Out of memory (OOM) or resource pressure.
- Daemon crash due to a bug or corrupt state.
- Systemd restart policy triggering a restart.
- Disk full (logs, images, containers).
- Kernel updates or host reboots.
- Configuration changes (e.g. /etc/docker/daemon.json).
- Security tools or process managers killing the daemon.
- Containerd or runc issues.
- Hardware or filesystem problems.
- SELinux/AppArmor policy conflicts.

INVESTIGATION STEPS

- 1 Check daemon logs
`journalctl -u docker --since "1 hour ago"`
- 2 Check system logs
`journalctl -xe | grep -i docker`
- 3 Check why the service restarted
`systemctl status docker`
- 4 Review system resource usage
`free -h && df -h && ps aux --sort=-%mem | head`
- 5 Check kernel messages
`dmesg | tail -n 50`
- 6 Look for OOM kills
`dmesg | grep -i kill`
- 7 Verify daemon configuration and recent changes
`cat /etc/docker/daemon.json`

KEY TAKEAWAYS

- Use systemd to manage and monitor the Docker daemon.
- Configure appropriate restart policies but investigate the root cause.
- Monitor host resources (CPU, memory, disk, inode).
- Keep Docker and host OS updated.
- Centralize and retain logs for analysis.
- Alert on daemon restarts.
- Test recovery procedures.



SENIOR INSIGHT

Frequent daemon restarts are a symptom, not the real problem. Always find and fix the root cause. Automated restarts can hide serious issues like memory leaks, disk exhaustion, or security conflicts.

DOCKER RUNTIME ARCHITECTURE (SIMPLIFIED)



USEFUL COMMANDS

- `systemctl status docker`
- `journalctl -u docker -n 100`
- `systemctl status containerd`
- `docker info` (if working)
- `ctr -n moby containers ls`
- `dmesg | tail -n 50`
- `free -h && df -h`
- `cat /etc/docker/daemon.json`

DISK AND FILESYSTEM EMERGENCY

Focus: safe investigation • images • volumes • logs • writable layers • destructive commands • recovery

SCENARIO 42

/var/lib/docker reaches 100 percent utilization during peak traffic.

WHAT IS HAPPENING?

- Docker storage at /var/lib/docker is full (100%).
- New containers may fail to start.
- Running containers can be affected (writable layers, logs, temp files).
- Images, containers, volumes, build cache, and logs consume space.
- High traffic can increase logs, temporary files, and writable layer growth.
- If the disk is full, the daemon and applications may become unstable.

USEFUL COMMANDS

```
df -h /var/lib/docker
docker system df -v
docker images
docker ps -s
docker volume ls
du -sh /var/lib/docker/containers/*
docker builder du
```

INVESTIGATION STEPS

- 1 Check disk usage
`df -h /var/lib/docker`
- 2 Find what is using space
`docker system df -v`
- 3 Check large images
`docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" | sort -h`
- 4 Check container writable layer size
`docker ps -s`
- 5 Check volume usage
`docker volume ls`
`docker system df -v | grep volume`
- 6 Check log file sizes
`du -sh /var/lib/docker/containers/*`
- 7 Check for build cache
`docker builder du`
- 8 Identify recent changes (deployments, large images, log growth)

KEY TAKEAWAYS

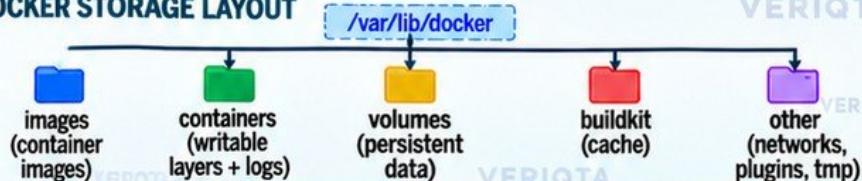
- /var/lib/docker stores images, containers, volumes, build cache, and logs.
- Monitor disk usage regularly.
- Use log rotation and set log limits for containers.
- Clean up unused images, containers, networks, and build cache.
- Consider separate disk or partition for Docker data.
- Set alerts before reaching critical usage (e.g. 80%).



SENIOR INSIGHT

Disk issues often start with log growth or unused images. Fix the root cause (e.g. log rotation, image cleanup policy), not just the symptom. Always investigate before removing anything.

DOCKER STORAGE LAYOUT



SCENARIO 43

An engineer proposes docker system prune -a on the affected production host.

WHAT DOES THIS COMMAND DO?

- `docker system prune -a` removes ALL unused resources:
 - All unused images (including unreferenced ones)
 - Stopped containers
 - Unused networks
 - Build cache
- It does NOT remove running containers or named volumes, but it can remove important images needed for rollback.
- It is destructive and cannot be easily undone.

WHAT SHOULD HAPPEN BEFORE RUNNING IT?

- 1 Assess the impact (what will be removed).
- 2 Confirm enough free space will be recovered (identify large unused images first).
- 3 Verify no running or stopped containers depend on the images.
- 4 Check if any rollback or disaster recovery images are needed.
- 5 Take a backup or snapshot if possible (VM snapshot or disk backup).
- 6 Consider safer, targeted cleanup instead of -a (e.g. remove specific images or cache).
- 7 Get approval from the team and follow change management process.

KEY TAKEAWAYS

- `docker system prune -a` is a last resort command.
- Always investigate and understand what is using disk space.
- Prefer targeted cleanup (remove specific images, old logs, or build cache).
- Ensure backups or snapshots exist before destructive operations.
- Have a recovery plan.
- Implement monitoring, log rotation, and image retention policies to prevent recurrence.



SENIOR INSIGHT

Never run `docker system prune -a` on a production host without fully understanding the impact. It can remove images needed for quick recovery and increase downtime.

PRODUCTION LOGGING FAILURE

Focus: ● logging drivers ● rotation ● stdout/stderr ● collectors ● backpressure ● disk impact ● observability architecture

SCENARIO 44

Container logs consume hundreds of gigabytes.

WHAT IS HAPPENING?

- Container logs are growing very large (hundreds of GB).
- Logs are written to stdout/stderr and stored by the Docker logging driver (usually json-file).
- No log rotation or retention policy is configured.
- Excessive logging (debug logs, repeated errors, high log volume).
- This can fill the /var/lib/docker filesystem and affect container and host stability.

INVESTIGATION STEPS

- 1 Check disk usage
`df -h /var/lib/docker`
- 2 Find largest log files
`du -sh /var/lib/docker/containers/*`
- 3 Identify noisy containers
`docker ps --format "table {{.ID}}t{{.Names}}t{{.Size}}"`
- 4 Inspect container logs
`docker logs --tail 100 <container>`
- 5 Check logging driver and settings
`docker inspect <container> --format '{{.HostConfig.LogConfig}}'`
- 6 Review application log level (debug vs info vs error)
- 7 Look for repeated errors or tight loops in application logs

KEY TAKEAWAYS

- Always configure log rotation for json-file driver.
- Avoid excessive debug logging in production.
- Monitor disk usage and log growth regularly.
- Use a centralized logging solution for long-term storage.
- Set appropriate log levels and structured logging.
- Large logs can cause disk full and service outages.



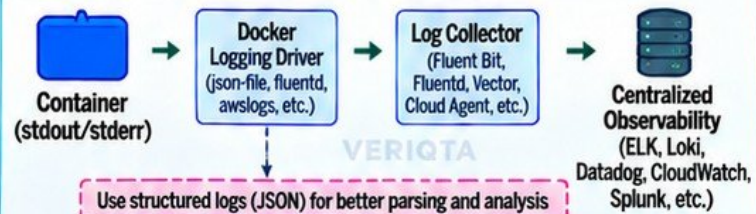
SENIOR INSIGHT

Logs are data. Treat them with the same lifecycle management as any other production data. Uncontrolled logs can silently take down your host. Implement rotation, retention and centralized logging early.

COMMON LOGGING DRIVERS

Driver	Description	Typical Use Case
json-file	Default driver (stores logs as JSON files)	Simple setups, small workloads
syslog	Sends logs to syslog server	Centralized logging
fluentd	Sends logs to Fluentd	Centralized logging
gelf	Sends logs to Graylog	Centralized logging
awslogs	Sends logs to AWS CloudWatch	AWS environments
none	Disables logging	Special cases (not recommended)

LOGGING ARCHITECTURE (EXAMPLE)



SCENARIO 45

docker logs works locally, but centralized observability has stopped receiving application logs.

POSSIBLE CAUSES

- Logging driver misconfigured.
- Log collector agent not running or crashed.
- Network connectivity issues between host and logging backend.
- Buffer or backpressure causing logs to be dropped.
- Filtering or parsing errors in collector.
- Logs sent to wrong endpoint, index, or stream.
- Permissions or authentication issues.

INVESTIGATION STEPS

- 1 Verify logs are still written locally
`docker logs --tail 100 <container>`
- 2 Check container logging configuration
`docker inspect <container> --format '{{.HostConfig.LogConfig}}'`
- 3 Check log collector agent status (systemctl status, logs)
- 4 Verify network connectivity to logging backend (curl, ping, firewall)
- 5 Check collector logs for errors (parsing, dropped logs, authentication)
- 6 Validate that the correct endpoint/index is configured
- 7 Send a test log and confirm it reaches the backend

KEY TAKEAWAYS

- Local logs working does not guarantee they are reaching your observability stack.
- Monitor your log pipeline (collectors, network, backend).
- Use health checks and alerting for log ingestion failures.
- Implement buffering and retry to handle backpressure.
- Use structured logging and consistent formats across services.
- Regularly test end-to-end log flow.



SENIOR INSIGHT

Observability is part of your production infrastructure. Treat log collection and forwarding as a critical service with monitoring, alerting and redundancy.

TRAFFIC SURGE AND CASCADING FAILURE

Focus: • saturation • connection pools • CPU throttling • retries • backoff • jitter • timeouts • cascading failure

SCENARIO 46

Traffic triples and containers remain running, but latency and errors explode.

WHAT IS HAPPENING?

- Incoming traffic has increased significantly (3x).
- Containers are still running, but the application is slowed down.
- Latency increases and error rates spike (e.g. 5xx, timeouts).
- Critical resources are saturated (CPU, memory, network, DB connections).
- The system has not scaled effectively to handle the load.
- If not addressed, this can lead to cascading failures across dependent services.

INVESTIGATION STEPS

- 1 Check resource usage (docker stats, top, metrics).
- 2 Identify which component is saturated (CPU, memory, I/O, network).
- 3 Check application metrics (latency, error rate, queue depth).
- 4 Inspect connection pool usage (DB, cache, external APIs).
- 5 Look for CPU throttling (cat /sys/fs/cgroup/cpu*/cpu.stat).
- 6 Check container logs for timeouts, rejections or rate limits.
- 7 Review auto-scaling settings (if using orchestrator).
- 8 Identify downstream dependency issues.

KEY TAKEAWAYS

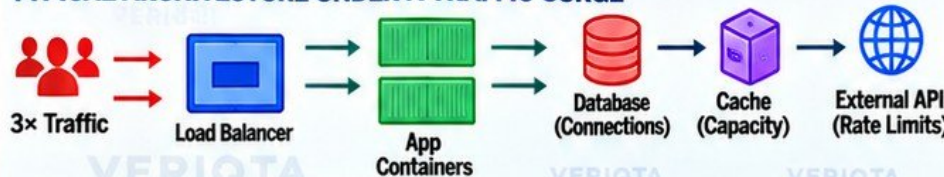
- Running containers does not mean the application is healthy.
- Monitor saturation early (CPU, memory, I/O, network, connections).
- Use proper auto-scaling and load balancing.
- Set realistic timeouts and limits.
- Configure connection pools correctly.
- Monitor end-to-end latency and error rates.
- Plan for traffic spikes (alerts, load tests, capacity planning).



SENIOR INSIGHT

Latency is an early signal of trouble. By the time you see errors, the system is already under significant stress. Monitor saturation, set proper limits, and scale before users feel the impact.

TYPICAL ARCHITECTURE UNDER A TRAFFIC SURGE



WHERE BOTTLENECKS HAPPEN

- CPU saturation (high CPU usage)
- Memory pressure (OOM, swapping)
- Network limits (bandwidth, connections)
- Database connection exhaustion
- Slow downstream services
- External API rate limiting
- Insufficient replicas or scaling delay

SCENARIO 47

Aggressive retries make a dependency outage significantly worse.

WHAT IS HAPPENING?

- A downstream dependency (e.g. database, external API) is down or experiencing issues.
- Clients retry aggressively without proper backoff or limits.
- The retries generate a large volume of additional traffic.
- This increases load on the already failing service, making recovery slower or impossible.
- It can also cause resource exhaustion in your own services (threads, connections, CPU).
- This creates a cascading failure across multiple services.

INVESTIGATION STEPS

- 1 Confirm the dependency is failing (check health, logs, status page).
- 2 Check retry configuration (retry count, backoff, timeout).
- 3 Analyze traffic patterns (metrics, logs).
- 4 Look for retry storms in application logs.
- 5 Check connection pool usage (are retries exhausting connections).
- 6 Identify impact on other services (cascading failures).
- 7 Implement mitigations (rate limiting, circuit breaker, reduce retries).

KEY TAKEAWAYS

- Retries without backoff can make outages worse.
- Use exponential backoff with jitter.
- Set reasonable retry limits and timeouts.
- Implement circuit breakers to fail fast.
- Monitor downstream dependency health.
- Use bulkheads and rate limiting to protect your system.
- Design for graceful degradation when dependencies are unavailable.

RETRY STRATEGY COMPARISON

Strategy	Description	Example	Best Practice
No backoff	Retries immediately	Retry every 1s	Avoid
Fixed backoff	Waits fixed time	Retry every 5s	Better, but can cause spikes
Exponential backoff	Increasing delay	1s, 2s, 4s, 8s	Recommended
Exponential + jitter	Randomized delay	1s, 2s, 4s ± jitter	Best practice



SENIOR INSIGHT

Retries are necessary, but they must be intelligent. Use exponential backoff with jitter, set limits, and implement circuit breakers. Protect your dependencies and your own services from retry storms.

DOCKER PRODUCTION ARCHITECTURE DECISIONS

Focus: • scheduling • reconciliation • HA • service discovery • scaling • deployment control • failure recovery • operational complexity

SCENARIO 48

A team proposes running a large production platform using manually managed standalone Docker hosts.

WHAT ARE THE OPERATIONAL CONCERNS?

- No automatic scheduling or placement of containers.
- No self-healing if a container or host fails.
- Manual scaling and load distribution.
- Service discovery must be built and maintained manually.
- Rolling updates and zero-downtime deployments are complex.
- High operational overhead (patching, monitoring, backups).
- Risk of configuration drift across hosts.
- No built-in high availability (HA).
- Harder to manage secrets, networking and access control.
- Limited visibility and centralized management.
- Becomes difficult to operate at scale.

INVESTIGATION / EVALUATION STEPS

- 1 Define platform requirements (availability, scale, performance).
- 2 Identify how containers will be scheduled and placed on hosts.
- 3 Determine failure recovery process (host or container failure).
- 4 Evaluate service discovery and internal networking.
- 5 Review deployment strategy (rolling updates, rollbacks).
- 6 Assess monitoring, logging and alerting approach.
- 7 Check how configuration, secrets and access will be managed.
- 8 Compare with an orchestrator (e.g. Kubernetes, Swarm, ECS) for gaps and risks.

KEY TAKEAWAYS

- Manually managed Docker hosts can work for small and simple setups, but are risky for large production platforms.
- Lack of automation increases operational effort and human error.
- As the platform grows, orchestration tools provide essential capabilities (scheduling, self-healing, scaling, HA, service discovery).
- Always assess long-term maintainability, not just short-term simplicity.



SENIOR INSIGHT

Running large production workloads on standalone Docker hosts is possible but rarely a good long-term choice. Use an orchestrator when you need reliability, scale, and reduced operational burden.

SCENARIO 49

When does Docker Compose remain appropriate, and when does the workload require stronger orchestration capabilities?

WHEN IS DOCKER COMPOSE APPROPRIATE?

- Local development and testing.
- Small applications or simple microservices.
- Non-critical or internal tools.
- Single host deployments.
- Environments where high availability is not required.
- Simple startup and shutdown workflow.
- Teams that need a lightweight and easy-to-use solution.
- Short-lived or temporary environments (e.g. demos, CI environments).

WHEN DO YOU NEED STRONGER ORCHESTRATION?

- 1 Multiple hosts or cloud infrastructure.
- 2 High availability and self-healing requirements.
- 3 Automatic scheduling and scaling.
- 4 Service discovery and advanced networking.
- 5 Rolling updates and deployment control.
- 6 Handling large numbers of services and containers.
- 7 Resource management (CPU, memory) across a cluster.
- 8 Production-grade observability, security and compliance.

KEY TAKEAWAYS

- Docker Compose is great for simple use cases and development.
- It is not designed for complex, multi-host, production-scale workloads.
- When you need automation, self-healing, scaling, or high availability, use an orchestrator such as Kubernetes, Docker Swarm or a managed platform (ECS, EKS, AKS, etc.).
- Choose the right tool based on current needs and future growth.

COMPOSE vs ORCHESTRATOR (HIGH-LEVEL)

Docker Compose
(Single Host)

docker-compose.yml

Start services

Manage locally

Good for simple apps

VS

Orchestrator
(Multi-Host)

Scheduler

Self-healing

Scaling

Service discovery

Production ready

FEATURE COMPARISON

Feature	Docker Compose	Orchestrator (Kubernetes/Swarm/ECS)
Host Support	Single host	Multi-host (cluster)
Scheduling	Manual	Automatic
Self-healing	No	Yes
Scaling	Manual (limited)	Automatic (horizontal/vertical)
Service Discovery	Basic (DNS)	Built-in (advanced)
Rolling Updates	Manual	Automatic
High Availability	No	Yes
Operational Complexity	Low	Higher (but necessary at scale)
Best Use Case	Development / small apps	Production / large scale

STAFF-LEVEL DOCKER INCIDENT

Focus: • end-to-end investigation • load balancer • networking • container networking • dependencies • systematic troubleshooting

SCENARIO 50

After a deployment, customers report intermittent 502 errors. Containers appear healthy, CPU is moderate, memory is below limits, and no obvious application exception exists. How would you lead the investigation?

UNDERSTAND THE PROBLEM

- Intermittent 502 errors after deployment.
- Containers appear healthy (running).
- CPU is moderate and memory is below limits.
- No obvious application exceptions.
- Could be network, load balancer, proxy, timeout, container networking, or dependency issue.
- Need a systematic, end-to-end investigation.

INVESTIGATION STEPS

- 1 Confirm the scope (which endpoints, users, regions, time pattern).
- 2 Check load balancer logs and metrics (5xx, target health, latency, timeouts).
- 3 Verify target/host health (security groups, firewalls, published ports).
- 4 Check container and Docker networking (netstat, docker ps, docker inspect).
- 5 Inspect application logs inside containers.
- 6 Check for timeouts between services (proxy, LB, app, dependency).
- 7 Verify service discovery / DNS resolution (container name, internal DNS).
- 8 Test dependencies (database, cache, external APIs).
- 9 Look for connection exhaustion (connection pools, max connections).
- 10 Correlate all logs and metrics to find the failure point.

KEY TAKEAWAYS

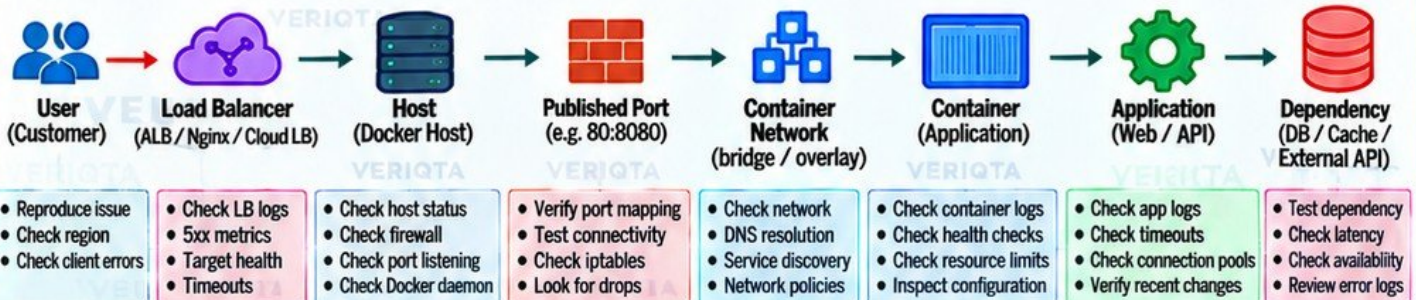
- A 502 error usually means a bad gateway (upstream issue).
- Healthy containers do not guarantee a healthy application.
- Check every layer from user to dependency.
- Look for timeouts, connection exhaustion, and network issues.
- Verify DNS, service discovery, and load balancer target health.
- Consider recent changes (deployment, config, network, security).
- Use metrics, logs, and tracing together.
- Intermittent issues often point to capacity, timeouts, or flaky dependencies.



SENIOR INSIGHT

502 errors are symptoms, not the root cause. Always investigate the full request path. The issue is often between components (proxy, load balancer, network, or dependency), not inside the container itself.

END-TO-END REQUEST FLOW (INVESTIGATION PATH)



USEFUL COMMANDS

- docker ps
- docker logs <container>
- docker inspect <container>
- docker network ls
- docker network inspect <network>
- ss -tulnp | grep <port>
- curl -v http://<host>:<port>
- nslookup <service>
- dig <service>
- netstat -anp

WHAT TO CHECK AT EACH LAYER

- 1 User – Can you reproduce? Any specific pattern?
- 2 Load Balancer – 5xx errors? Target health? Timeouts?
- 3 Host – Is the service listening? Any network blocks?
- 4 Published Port – Is the port reachable externally?
- 5 Container Network – Is DNS working? Can containers reach each other?
- 6 Container – Are logs clean? Any restarts? Health status?
- 7 Application – Any slow responses? Check upstream calls.
- 8 Dependency – Is the database/cache/API healthy?

COMMON ROOT CAUSES

- Load balancer timeout too short
- Unhealthy targets behind the LB
- Connection pool exhaustion
- Slow or failing dependency
- DNS resolution issues
- Network policy or firewall blocking
- Resource limits (ephemeral ports, file handles)
- Recent deployment or configuration change
- Intermittent network instability
- Service discovery registration delays

RESOLUTION APPROACH

- ✓ Identify the failing component
- ✓ Fix the root cause (not just the symptom)
- ✓ Add monitoring and alerts
- ✓ Implement preventive measures (timeouts, retries, circuit breakers, capacity)